ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

On Combiners for Verifiable Random Functions

Bachelor Thesis

Basil Rohner

March 17, 2026

Advisors: Prof. Dr. A. D. Visor, Dr. P. Ostdoc
Department of Computer Science, ETH Zürich

Acknowledgments

Abstract

Contents

Contents **iii**

1 Introduction **1**
1.1 Motivation and Previous Work 1
1.2 PRF Combiners 3
1.3 Our Contribution 4

2 Notation and Preliminaries **7**
2.1 Notation 7
2.2 Verifiable Functions 8
2.3 Robust Combiners 11

3 Results **13**
3.1 Robust Black-Box DAG Combiner 13
3.2 Impossibility of Robust Black-Box VRF-Combiner 18
3.3 Strong Robust Black-Box Combiners 30
3.4 Discussion and Extensions 33

4 Conclusion **34**

Appendix **35**
The Difference Lemma 35

Chapter 1

Introduction

1.1 Motivation and Previous Work

Pseudorandom Functions. A pseudorandom function (PRF) [**?**] is a deterministic algorithm that takes as input a secret key k and a point x to produce an output y that looks random in the sense that no efficient adversary, modeled by a probabilistic polynomial time (ppt.) Turing machine can distringuish the output y from a random bitstring r of the same length. This must hold even if the adversary has access to a polynoial number of point evaluations (x, y) before distinguishing the output. PRFs are fundamental to modern cryptography and used to construct ....

Verifiable Functions. A verifiable function (VF) is a public-key primitive that allows the holder of a secret key sk to evaluate points x to obtain image-proof pairs (y, π) such that any party holding the public key pk can verify that y was indeed computed correctly from x if given the proof π. We require correctness and unique provability from the verification which state that correctly generated image-proof pairs (y, π) always (or except with negligible probability) is accepted by the verification algorithm and that at most one image can be proven respectively.

In this work we primarily study extensions of VRs, primarily the verifiable random function (VRF) introduced by Micali, Rabin and Vadhan in [**?**] that additionally requires the outputs y to be pseudorandom to efficient adversaries without access to the corresponding proof. Similar to the pseudorandomness definition for PRFs we require that no efficient adversary can distinguish the images y from random outputs r, even if the adversary gets to see polynomially many evaluations of x with their corresponding proof. VRFs can be seen as a public-key analogoue of PRFs with verifiability requirements.

In practice VRFs are used to implement signatures with strong verifiability guarantees [**?**], resettable zero-knowledge proofs [**?**], lottery and e-cash sys-

tems [?] and they play a big role in distributed leader election protocols that runs many Proof-of-Stake blockchain systems [?].

Additionally we study VFs satisfying weaker notions of pseudorandomness such as weak pseudorandomness and unpredictability which give rise to verifiable unpredictable functions (VUFs) [?] and weak verifiable random functions (wVRFs) [?] respectively. VUFs additionally require that the evaluation of the VF on a fresh evaluation point cannot be predicted by efficient adversaries except with negligible probability and wVRFs require that pseudorandomness holds, but only on uniformly random evaluation points x, instead of a point chosen by the adversary.

add stronger notions cVRF, khVRF, aggVRF, unbiasVRF if time

Robust Combiners. In cryptography we often take a reductionist approach when constructing new cryptographic primitives, in general constructions satisfying computational security properties rely on cryptographic hardness assumptions. To hedge against the breakdown of cryptographic hardness assumptions, and thus the subsequence failure of the construction we may want to base the security of systems on multiple hardness assumptions. Robust combiners [?] are constructions that combine n candidate schemes, concrete instances of the same primitive, and implement a secure instance of that primitive if a large enough subset of those candidate is secure. The notion of combiners has gained traction with the rise of post-quantum and hybrid-cryptography areas

Stronger version of robust combiners are robust black-box combiners [?] which are only given query access to the candidate schemes as oracles, i.e. cannot inspect the Turing machine of the candidate scheme. The motivation of a robust black-box combiner is to have a combiner that can implement a secure scheme only with access to sub-schemes with the underlying code of the candidate schemes are not accessible to the combiner. We wish for the combiner to derive its security from the evaluation of the secure subset of oracles, not by making any modifications to the candidates or simply implementing a secure instance of the primitive himself. For that reason we additionally require robust black-box combiners to implement the secure instance relative to a PSPACE complete oracle, that is guarantee the security against adversaries with access to a PSPACE oracle.

Robust black-box combiners have been considered for a variety of cryptographic primitives such as oblivious transfer [?], non-interactive zero-knowledge proofs (NIZK) [?], non-interactive witness-indistinguishable arguments (NIWI) [?], indistinguishability obvuscation (iO) [?] and signature schemes [?]. Additionally many folklore constructions exist for PRF, key exchange protocols . While there exist VRF constructions rely on PRFs and NIWIs, for both of which combiners have been studied, black-box combiners for VRFs have, to the best of our knowledge, not been studied.

add some

1.2 PRF Combiners

VRF can be considered as an extension of PRFs to the public-key setting with the additional requirements of verifiability. Simple robust black-box PRF combiners exist. One combiner works by evaluating the evaluation point x with every candidate scheme to obtain images y_1, y_2 and returning the XOR of the two images $y = y_1 \oplus y_2$. Pseudorandomness of the combined scheme can be shown with a reductionist approach. Specifically we show that given an adversary against the combined scheme, we can construct an adversary against every candidate scheme running in the same time with the same advantage. By contraposition, assuming that at least one candidate scheme is pseudorandom and there exists no efficient combiner with non-negligible advantage against the candidate we show that there exists no efficient adversary against the combined scheme with non-negligible advantage. We illustrate the construction of a robuts $(2;1)$ black-box PRF-combiner in Section 1.2.

$f^{\mathcal{O}_1,\mathcal{O}_2}(k,x)$

1 : $y_1 \leftarrow \mathcal{O}_1(k,x)$
2 : $y_2 \leftarrow \mathcal{O}_2(k,x)$
3 : **return** $y_1 \oplus y_2$

Figure 1.1: Robust $(2;1)$ black-box PRF-combiner.

We outline the reduction and refer to the appendix for the formal proof. The reduction against candidate scheme $\mathcal{O}_i$ relies on the pseudorandomness oracle $\mathcal{O}_i^{prf}$ and simulates the other candidate $\mathcal{O}_{2-i}^{prf}$ himself to answer queries of the combiners adversary with $\mathcal{O}_i^{prf}(x) \oplus \mathcal{O}_{2-i}(k,x)$. The adversary against the combined scheme then guesses the hidden bit b, trying to distinguish the random world from the real world which the reduction then uses to guess its own hidden bit b'. If the hidden bit $b' = 0$ then the adversary $\mathcal{A}$ is simulated perfectly in the sense that the reduction wins the security experiment with the same advantage of $\mathcal{A}$ while for bit $b' = 1$ the answers to the queries of the adversary are still uniformly random bitstrings and therefore the reduction also wins with the same probability.

Note that the existance of a robust $(2;1)$ black-box PRF-combiner naturally leads to a robust $(n;1)$ black-box VRF-combiner for any natural number n by iteratively applying the construction on the construction and a new candidate. Since there exists a robust $(n;1)$ black-box PRF-combiner one could hope for a similar construction to work for VRFs. This is however not the case, as we will show in Theorem 3.9 there exist no robust $(n;\lfloor n/2 \rfloor)$ black-box combiner for VRFs.

1.3 Our Contribution

Robust Black-Box DAG Combiner. We introduce the notion of robust black-box DAG combiners, formally defined in Definition 3.1, DAG combiners for short. DAG combiners are black-box combiners which are defined by directed acyclic graphs (DAGs) for every implemented algorithm. The implemented algorithms are specified through computation that runs along the DAG starting from a designated source vertex and ending in a target vertex. In the execution vertices correspond to queries to implementation functions of candidate oracles on an input computed from all preceding vertices by a ppt. Turing machine (without oracle access to the candidates). DAG combiners make the structure of the black-box combiner more explicit by structuring it into interleaved oracle queries to the candidats and intermediate computation, that we model with Turing machines without oracle access to the candidates. We illustrate this concept in Figure 1.2 on the example of the simple robust black-box PRF-combiner in Section 1.2.

To connect DAG combiners and general black-box combiners we show that the notions are indeed equivalent, that is, that for every general black-box combiner we can construct a DAG combiner that behaves identically in the sense that the algorithms defined by the combiners are equivalent. We state this result formally in Theorem 3.2. The concept of a robust black-box DAG combiner allows us to reason over a more structured representation of robust black-box combiners, more similar to a circuit in structure simplifying the proofs later on.

Robust Black-Box VRF-Combiner. Following the introduction of a robust black-box DAG combiner, we study the simplest robust black-box VRF-combiner, robust $(2;1)$ black-box VRF-combiners. We answer the question on whether there is a construction that combines two candidates, of which only at least one is secure, negatively. Specifically we show impossibility of robust $(2;1)$ black-box VRF-combiners in Lemma 3.3.

Since we can naturally use a robust $(n;\lfloor n/2 \rfloor)$ black-box VRF-combiner to construct a $(2;1)$ combiner by simply initializing half the candidates with $\mathcal{O}_1$ and the other half with $\mathcal{O}_2$ we get an impossibility result for robust $(n;\lfloor n/2 \rfloor)$ black-box VRF-combiner by contraposition. Next we inspect robust black-box combiners with additional assumptions on the candidate schemes and relaxed requirements on the construction in the hopes of finding combiner constructions. We show that even if we only require unpredictability, statistical correctness and statistical unique provability from the construction, the impossibility holds Theorem 3.9, this gives two corollaries excluding the possibility of robust $(n;\lfloor n/2 \rfloor)$ black-box combiners for wVRF and VUF in Corollary 3.10 and Corollary 3.11. By strengthening the assumptions on

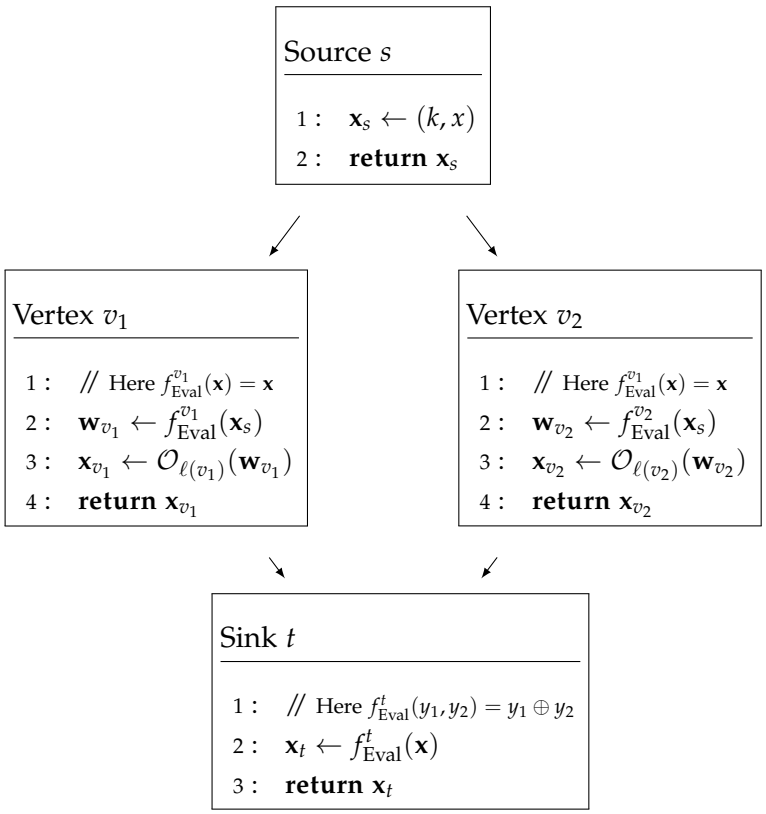


Figure 1.2: Robust $(2;1)$ Black-Box DAG PRF-combiner. Note that for every node we first combine the output of all predecessors using a ppt. Turing machine f^{v}_{Eval}. The output $\mathbf{w}_v$ of f^{v}_{Eval} is then queried to the corresponding oracle $\mathcal{O}_{\ell(v)}$. The source vertex doesn't compute anything but simply outputs the inputs while the sink t doesn't query any oracle and simply combines its inputs.

the candidates for all candidates to implement pseudorandomness is also insufficient to find constructions Theorem 3.9.

unclear

We show that if we assume all candidates to be uniquely provable and perfectly correct, the combination of the candidates works in a very similar fashion to the combiner on the PRF primitive. Note that since unique provability and perfect correctness are generally information theoretic properties, this means that for most VRF constructions, e.g. LatticeVRF [**?**] and ECVRF [**?**], those properties are typically proven mathematically. Arguable combiners are only interesting for combining computational security properties that rely on hardness assumptions.

Having strengthened the assumptions on the candidates we can now consider more advanced notions of VRFs. We'll show that with simple extensions we can construct combiners for constrainable VRFs, key-homomorphic VRFs and

aggregable VRFs if all candidates are guaranteed to be uniquely provable and perfectly correct. Finally we consider unbiasable VRFs for which we provide a combiner that relies on collision resistant hash functions and deterministic 2-source randomness extractors.

Chapter 2

Notation and Preliminaries

2.1 Notation

For a positive integeger n define $[n] = \{1, \ldots, n\}$. We use $\lambda \in \mathbb{N}$ exclusively to denote the security parameter. We use $\mathsf{poly}(\lambda)$ to denote the set of functions that are polynomial in λ in the sense of Definition 2.1 and $\mathsf{negl}(\lambda)$ to denote the set of functions that are negligible in λ in the sense of Definition 2.2. We call a function f *overwhelming* if $1 - f(\lambda) \in \mathsf{negl}(\lambda)$. We use $\leftarrow$ for simple assignment. For a set S we write $s \leftarrow_\$ S$ to indicate that the value s is sampled uniformly at random from set S and for a probabilistic algorithm $\mathcal{A}$ we write $y \leftarrow_\$ \mathcal{A}(x)$ to denote assigning to y the output of $\mathcal{A}$ on input x. We use ppt. to abbreviate probabilistic polynomial time in the context of Turing machines. Finally, we use bold letters, e.g. $\mathbf{x}$, to indicate that $\mathbf{x}$ consists of multiple values such as tuples or vectors.

maybe change

Definition 2.1 (Polynomial Functions) *We say that a function $p : \mathbb{N} \to \mathbb{R}$ is polynomial if the folowing holds*

$$\exists d \in \mathbb{N}, \exists n_0 \in \mathbb{N}, \forall n > n_0, p(n) \leq n^d$$

Definition 2.2 (Negligible Functions) *We say that a function $\varepsilon : \mathbb{N} \to \mathbb{R}$ is negligible if the following holds*

$$\forall p \in \mathsf{poly}(\lambda), \exists n_0 \in \mathbb{N}, \forall n > n_0, |\varepsilon(n)| < \frac{1}{p(n)}$$

2.2 Verifiable Functions

A verifiable function (VF) VF is a cryptographic primitive that implements algorithms $\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}$ and $\mathsf{Vfy}_{\mathsf{VF}}$. A user can generate key-pairs $(\mathsf{sk}, \mathsf{pk})$ using $\mathsf{Gen}_{\mathsf{VF}}$ consisting of a secret key sk and a public key pk, use the secret key sk to make evaluate arbitrary points $x \in \{0,1\}^\lambda$ using $\mathsf{Eval}_{\mathsf{VRF}}$ and use the public key pk to verify a triple (x, y, π) using $\mathsf{Vfy}_{\mathsf{VRF}}$. We give a formal definition in Definition 2.3, the definition takes inspiration of the definition of a VRF in [**?**]. To reduce redundancy we define additional security properties such as pseudorandomness and unpredictability seperately.

Definition 2.3 (Verifiable Function) *A verifiable function (VF)* VF *is a triple of efficient (ppt.) algorithms* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *with the syntax*

this is less elegant now

- $\mathsf{Gen}_{\mathsf{VF}}$*: The key-generation algorithm takes as input the unary encoding of the security parameter and produces a pair* $(\mathsf{sk}, \mathsf{pk}) \in \mathcal{SK} \times \mathcal{PK}$ *containing a secret and a public key respectivley.*
- $\mathsf{Eval}_{\mathsf{VF}}$*: The evaluation algorithm takes as input a tuple* (sk, x) *containing a secret key* $\mathsf{sk} \in \mathcal{SK}$ *and an evaluation point* $x \in \{0,1\}^\lambda$ *and produces an tuple* (y, π) *containing an image* $y \in \mathcal{Y}$ *and a proof* $\pi \in \mathcal{P}$*.*
- $\mathsf{Vfy}_{\mathsf{VF}}$*: The verification algorithm takes as input a tuple* (pk, x, y, π) *containing a public key* $\mathsf{pk} \in \mathcal{PK}$*, an evaluation point* $x \in \{0,1\}^\lambda$*, an image* $y \in \mathcal{Y}$ *and a proof* $\pi \in \mathcal{P}$ *and returns a bit b denoting acceptance or rejection.*

can evaluation or verification be randomized, probably vfy can be probabilistic

Note that a VF is a purely syntactical concept. For a VF we define various properties to arrive to arrive at specific cryptographic primitives, e.g. VRFs. First of all we define ε-correctness, a property that guarantes that over random key-pairs and evaluation points, the verification algorithm accepts a correctly computed image-proof pair with probability at least ε.

Definition 2.4 (ε-Correctness, [?]) *A verifiable function* $\mathsf{VF} = (\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is ε-correct, if for all security parameters $\lambda \in \mathbb{N}$ we have that*

$$\Pr\left[\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y, \pi) = 1 : \begin{array}{c}(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^\lambda) \\ x \leftarrow_{\$} \{0,1\}^\lambda \\ (y, \pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)\end{array}\right] \geq \varepsilon(\lambda)$$

We define a VF VF *to be perfectly correct, or just correct, if it is* 1*-correct and statistically correct if ε is an overwhelming function.*

Intuitively a correct VF states that honestly generated image-proof pairs are accepted by the verification algorithm. Next we define unique provability. A VF is uniquely provable if not more than one image can be proven, even for maliciously crafted public keys.

Definition 2.5 (Unique Provability, [?]) *A verifiable function* VF*, with algorithms* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$*, is defined to be uniquely provable if for all evaluation points* $x \in \{0,1\}^\lambda$*, all public-keys* $\mathsf{pk} \in \mathcal{PK}$ *and any two images-proof pairs* $(y_0, \pi_0), (y_1, \pi_1) \in \mathcal{Y} \times \mathcal{PK}$ *we have:*

$$\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y_0, \pi_0) = 1 \wedge \mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y_1, \pi_1) = 1 \implies y_0 = y_1$$

maybe comment why no statistical notion

Correctness and unique provability are information theoretic. We define pseudorandomness of a VF which is a computational property, stating that no efficient adversary $\mathcal{A}$ can distinguish images y corresponding to evaluation of x, chosen by $\mathcal{A}$, from a uniformly random output $y^* \leftarrow_\$ \mathcal{Y}$. This needs to hold even if the adversary gets to see the evaluation of a polynoial number of points $x \neq x^*$.

Definition 2.6 ((t,ε)-Pseudorandomness, [?]) *A VF* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is defined to be* (t,ε)*-pseudorandom if for all security parameters* $\lambda \in \mathbb{N}$*, for all stateful adversaries* $\mathcal{A}$*, running in time at most* $t(\lambda)$*, the advantage of* $\mathcal{A}$ *in the prf security experiment outlined in Figure 2.1 is at most* $\varepsilon(\lambda)$*, i.e. if*

$$\mathbf{Adv}^{\mathrm{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

We say that a VF is pseudorandom, if it is (t,ε)*-pseudorandom with* ε *being a negligible function and* t *being a polynomial, i.e.* $\varepsilon \in \mathsf{negl}(\lambda), t \in \mathsf{poly}(\lambda)$*.*

Next we define two weaker notions of pseudorandomness. While (strong) pseudorandomness required the evaluation of adverserially chosen points to be indistinguishable from uniformly random elements $y^* \in \mathcal{Y}$ we can relax this requirement by requiring computational indistinguishability for uniformly random challenges $x^* \leftarrow_\$ \{0,1\}^\lambda$. We call this notion weak pseudorandomness.

Definition 2.7 ((t,ε)-Weak Pseudorandomness, [?]) *A VF* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is defined to be* (t,ε)*-weakly pseudorandom if for all security parameters* $\lambda \in \mathbb{N}$*, for all stateful adversaries* $\mathcal{A}$*, running in time at most* $t(\lambda)$*, the advantage of* $\mathcal{A}$ *in the wprf security experiment outlined in Figure 2.1 is at most* $\varepsilon(\lambda)$*, i.e. if*

$$\mathbf{Adv}^{\mathrm{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

We say that a VF is weakly pseudorandom, if it is (t,ε)*-weakly pseudorandom with* ε *being a negligible function and* t *being a polynomial, i.e.* $\varepsilon \in \mathsf{negl}(\lambda), t \in \mathsf{poly}(\lambda)$*.*

Alternatively a VF can be unpredictable. A VF is unpredictable if no adversary can predict the image y of the evaluation of a challenge x chosen by the adversary. This property needs to hold even if the adversary sees the evaluation of a polynomial number of points $x \neq x^*$. Note that if the adversary can predict the output of evaluation on x^* he can also distinguish the output from a random output $y^* \leftarrow_{\$} \{0,1\}$ with non-negligible probability making unpredictability is a weaker property than pseudorandomness.

Definition 2.8 ((ε, t)-Unpredictability, [?]) *A VF $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ is defined to be (t, ε)-unpredictable if for all security parameters $\lambda \in \mathbb{N}$, for all stateful adversaries $\mathcal{A}$, running in time at most $t(\lambda)$, the advantage of $\mathcal{A}$ in the up security experiment outlined in Figure 2.1 is at most $\varepsilon(\lambda)$, i.e. if*

$$\mathbf{Adv}^{\mathrm{up}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{up}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

We say that a VF is unpredictable, if it is (t, ε)-unpredictable with ε being a negligible function and t being a polynomial, i.e. $\varepsilon \in \mathsf{negl}(\lambda), t \in \mathsf{poly}(\lambda)$.

We now define the notions of a verifiable random function (VRF), verifiable unpredictable function (VUF) and weak verifiable random function (WVRF), which are correct, uniquely provable and pseudorandom, unpredictable and weakly pseudorandom VFs respectivly. We formally define the primitives.

Definition 2.9 (Verifiable Random Functions) *A verifiable random function $\mathsf{VRF} = (\mathsf{Gen}_{\mathsf{VRF}}, \mathsf{Eval}_{\mathsf{VRF}}, \mathsf{Vfy}_{\mathsf{VRF}})$ is a VF that satisfies perfect correctness in the sense of Definition 2.4, unique provability in the sense of Definition 2.5 and pseudorandomness in the sense of Definition 2.6.*

Definition 2.10 (Weak Verifiable Random Functions) *A weak verifiable unpredictable function $\mathsf{WVRF} = (\mathsf{Gen}_{\mathsf{WVRF}}, \mathsf{Eval}_{\mathsf{WVRF}}, \mathsf{Vfy}_{\mathsf{WVRF}})$ is a VF that satisfies perfect correctness in the sense of Definition 2.4, unique provability in the sense of Definition 2.5 and weak pseudorandomness in the sense of Definition 2.7.*

Definition 2.11 (Verifiable Unpredictable Functions) *A verifiable unpredictable function $\mathsf{VUF} = (\mathsf{Gen}_{\mathsf{VUF}}, \mathsf{Eval}_{\mathsf{VUF}}, \mathsf{Vfy}_{\mathsf{VUF}})$ is a VF that satisfies perfect correctness in the sense of Definition 2.4, unique provability in the sense of Definition 2.5 and unpredictability in the sense of Definition 2.8.*

Experiment $\mathbf{Exp}^{\text{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $(x^*, \sigma) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
6 : $y_1 \leftarrow_{\$} \mathcal{Y}$
7 : $b' \leftarrow_{\$} \mathcal{A}(1^{\lambda}, x^*, \sigma, y_b)$
8 : **return** $b = b' \wedge x^* \notin S$

Experiment $\mathbf{Exp}^{\text{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $\sigma \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $x^* \leftarrow_{\$} \{0,1\}^{\lambda}$
6 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
7 : $y_1 \leftarrow_{\$} \mathcal{Y}$
8 : $b' \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^{\lambda}, x^*, \sigma, y_b)$
9 : **return** $b = b' \wedge x^* \notin S$

Experiment $\mathbf{Exp}^{\text{up}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
2 : $S \leftarrow \varnothing$
3 : $(x^*, y^*) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^{\lambda}, \mathsf{pk})$
4 : $(y, \pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
5 : **return** $y^* = y \wedge x^* \notin S$

Oracle $\mathcal{O}_{\text{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)$
3 : **return** y

Figure 2.1: The pseudorandomness security experiment $\mathbf{Exp}^{\text{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ used in Definition 2.6, the weak pseudorandomness security experiment $\mathbf{Exp}^{\text{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ used in Definition 2.7 and the unpredictability security experiment $\mathbf{Exp}^{\text{up}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ used in Definition 2.8. The main changes between the pseudorandomness security experiment and the weak pseudorandomness security experiment are highlighted in grey.

2.3 Robust Combiners

For two natural numbers n, k we define the robust $(n;k)$ black-box combiner, or robust black-box combiner for short. A robust $(n;k)$ black-box combiner for a primitive P is a construction Π_{P}, to be precise a ppt. oracle Turing machine, that is given query access to n oracles, the candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$. If at least k candidate schemes implement the primitive P, then the combiner $\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}$ securely implements P. The idea behind a robust black-box combiner is that the construction extracts the security properties from the subset of correct candidates, even if some candidate schemes are not secure instances. However now note the following problem in the current definition:

If there exists a secure instance of P there always exists robust $(n;0)$ black-box combiner that simply ignores its candidates and implements P. This robust black-box combiner meets all the requirements of the definition. Showing

that there exists no specific robust black-box combiner therefore requires to show that no secure instance of the primitive P exists, which is often a difficult complexity theoretic statement.

To circumvent this problem we want to explicitly force the combiner to derive its security from the subset of secure candidate schemes. To do this, we require the security properties of the combiner to hold relative to a PSPACE complete oracle.

We write robust combiner for robust $(n;k)$ P-combiner whenever n, k and P are clear from the context. A robust combiner gets the candidate schemes as a description of their turing machine and can therefore be adaptive in the code of its candidates. Note that this definition allows the robust combiner to ignores its candidate schemes and implement a secure instance of the underlying primitive indepdently of its candidates. One can therefore always construct a robust combiner for primitives that admit secure instances, by contraposition this also means that any result that rules out the existence of a robust combiner for a specific primitive P also rules out the existence of the primitive itself. Intuitively we would like for the combiner to derive its security from the security of its secure candidate schemes, therefore we define robust black-box combiner. A robust black-box combiner is a robust combiner that, instead of getting access to the description of the Turing machines of the candidate schemes, only gets oracle access to the underlying candidate schemes. We define the notion of a robust black-box combiner closely following the definition by [**?**]. Note that we explicitly require a robust black-box combiner to derive its security from its candidates, for that reason we require the security to hold relative to a PSPACE oracle PSPACE.

Very unhappy with this definition

Definition 2.12 ($(n;k)$-Robust Black-Box P-Combiner) *Given a cryptographic primitive* P *consisting of p algorithm* $(\mathsf{alg}_\mathsf{P}{}^1, \ldots, \mathsf{alg}_\mathsf{P}{}^p)$*, a robust* $(n;k)$ *black-box* P*-combiner is a p-tuple of oracle Turing machines with access to n oracles, the candidate schemes* $\mathcal{O}_1, \ldots, \mathcal{O}_n$

$$\Pi_\mathsf{P}^{\mathcal{O}_1,\ldots,\mathcal{O}_n} = \left(\Pi_\mathsf{P}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}.\mathsf{alg}_\mathsf{P}{}^1, \ldots, \Pi_\mathsf{P}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}.\mathsf{alg}_\mathsf{P}{}^n\right)$$

A $(n;k)$*-robust combiner* $\Pi_\mathbf{P}$ *for primitive P is defined to be black-box, denoted by* Π_P*, if additionally it satisfies that if there exists an adversary* $\mathcal{A}$ *(possibly computationally unbounded) that breaks the security of* $\Pi_\mathsf{P}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$*, then there exist adversaries* $\mathcal{B}_1, \ldots, \mathcal{B}_{n+k-1}$ *with oracle access to* $\mathcal{A}$ *for at least* $n-k+1$ *different candidates.*

We remark that compositite security definitions that rely on multiple properties are broken if there exists an adversary that breaks a security property with significant advantage. Note that if the security would not hold relative to a PSPACE oracle PSPACE then we would find that a secure implementation

of the primitive would indeed implement a secure robust black-box combiner since there exists no adversary breaking the security properties of P.

Chapter 3

Results

3.1 Robust Black-Box DAG Combiner

Intuitively, a robust black-box combiner operates on n candidate schemes by interleaving oracle calls to the candidates implemented functions with processing to combine their outputs in a way such that security properties of the secure subset of oracles are preserved if there are sufficiently many of them. To argue about the impossibility of such constructions it is helpful to impose additional structure on the combiner, to structure it as a directed acyclic graph modeling the computation of the combiner. In the DAG every oracle query corresponds to a vertex colored with the appropriate label Viewing a robust black-box combiner as a robust black-box DAG combiner is therefore simply a way make the structured execution of a combiner explicit, making the reasoning about oracle query dependencies and oracle-independent computation explicit. Additionally since robust black-box combiners need to satisfy the security properties relative to a PSPACE oracle, in the later proof we rely on the ability of the PSPACE oracle to brute-force many secret keys in the algorithm if there are not queries to oracles. For this line of reasoning this description is also more convenient. As we will show later in Theorem 3.2, the notions of robust black-box combiners and robust black-box DAG combiners are equivalent.

We define robust black-box DAG combiners by representing each implemented algorithm alg with a directed acyclic graph $\mathcal{G}_{\mathsf{alg}} = (V_{alg}, E_{alg})$ with a source $s_{\mathsf{alg}} \in V_{\mathsf{alg}}$ that has no in-going edges, corresponding to the input of the algorithm, and a sink $t_{alg} \in V_{alg}$ with no out-going edges, corresponding to a final processing step. Additionally the description of the DAG combiner contains a vertex labling $l_{alg} : V_{alg} \setminus \{s_{alg}, t_{alg}\} \to [n] \times \{alg_1, \ldots, alg_k\}$ which maps every vertex that is not a source or a sink to the corresponding candidate oracle and the specific function called on an input $\mathbf{x}_v$ to produce an output $\mathbf{y}_v$. We define the output $\mathbf{y}_{s_{alg}}$ of the source to be the combiner

Algorithm $[\![\mathcal{G}, c, s, t, \{f_v\}_{v\in V(\mathcal{G})}; \rho]\!](\mathbf{x})$

$\mathbf{y}_s \leftarrow \mathbf{x}$
for $v \in \tau(\mathcal{G}, s) \setminus \{s, t\}$ **do**
  $\mathbf{x}_v \leftarrow f_v(\mathbf{x}, \{(\mathbf{y}_w, w)\}_{w\in\mathcal{N}^-(v)}; \rho)$
  $\mathbf{y}_v \leftarrow \mathcal{O}_{c(v)}(\mathbf{x}_v)$
return $f_t(\mathbf{x}, \{\mathbf{y}_w\}_{w\in\mathcal{N}^-(t)})$

Algorithm $\tau(\mathcal{G}, s)$

// Topological ordering TODO

Figure 3.1: Interpretation of $\left(\mathcal{G}, c, s, t, \{f_v\}_{v\in V(\mathcal{G})}\right)$ into an algorithm. τ returns the vertices in $\mathcal{G}$ in topological order starting from the source s.

algorithm input and the output of the sink t_{alg} to be the output of the combiner. The input $\mathbf{x}_v$ to the query at vertex $v \in V_{alg} \setminus \{s_{alg}, t_{alg}\}$ is computed by an efficient (ppt.) Turing machine f^v_{alg} associated with vertex v in G_{alg} which takes as input the labeled outputs of all predecessors $w \in \mathcal{N}_{in}(v)$ and produces an oracle query $\mathbf{x}_v$. We formally describe robust black-box DAG combiners in the following definition.

Definition 3.1 (Robust $(n;k)$ Black-Box DAG P-Combiner) *Given a cryptographic primitive* P *implementing a set of algorithms indexed by $j \in [|\mathsf{P}|]$. A robust $(n;k)$ black-box DAG* P*-combiner is a $|\mathsf{P}|$-tuple*

$$\left(\mathfrak{G}_j = \left(\mathcal{G}_j, c_j, s_j, t_j, \{f_{j,v}\}_{v\in V(\mathcal{G}_j)}\right)\right)_{j\in[|\mathsf{P}|]}$$

where $\mathcal{G}_j$ is a directed acyclic graph with vertex set $V(\mathcal{G}_j)$, $c_j : V(\mathcal{G}_j) \setminus \{s_j, t_s\} \to [n] \times [|\mathsf{P}|]$ is a vertex-coloring with colors (a, b) indicating that the corresponding query is made to algorithm a of candidate oracle b, source and sink $s_j, t_s \in V(\mathcal{G}_j)$ satisfying $\mathcal{N}^-(s_j) = \mathcal{N}^+(t_j) = \varnothing$ and for simplicity the only vertices satisfying those properties and finally $\{f_{j,v}\}_{v\in V(\mathcal{G}_j)}$ are ppt. Turing machines. satisfying that the following tuple of oracle Turing machines is a robust $(n;k)$ black-box combiner.

$$\widetilde{\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}} = \left(\widetilde{\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}}.[\![\mathfrak{G}_1]\!], \ldots, \widetilde{\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}}.[\![\mathfrak{G}_{|\mathsf{P}|}]\!]\right)$$

Where $[\![\mathfrak{G}_j]\!]$ is the interpretation according to Figure 3.1.

A robust black-box DAG combiner algorithm is then executed by tracing the computation through the corresponding DAG G_{alg}, starting at the source s_{alg} and ending at the sink t_{alg} in a topological ordering. The interpretation of an algorithm alg described by the robust black-box DAG combiner by $(\mathcal{G}_{alg}, s_{alg}, t_{alg}, l_{alg}, \{f^v_{alg}\}_{v\in V_{alg}\setminus\{s_{alg},t_{alg}\}})$ is shown in Figure 3.1.

Example. We introduce the notion of robust black-box DAG combiners to make the underlying structure of robust black-box combiners more explicit. To illustrate the definition we will show the structure of the DAG combiner of a robust $(2;1)$ black-box combiner **??** in Section 3.1. We label every vertices v in the DAGs that is not a source or sink with $l_{alg}(v)$, i.e. their vertex labling and for every vertex add the functional description of the Turing machines f^v_{alg}.

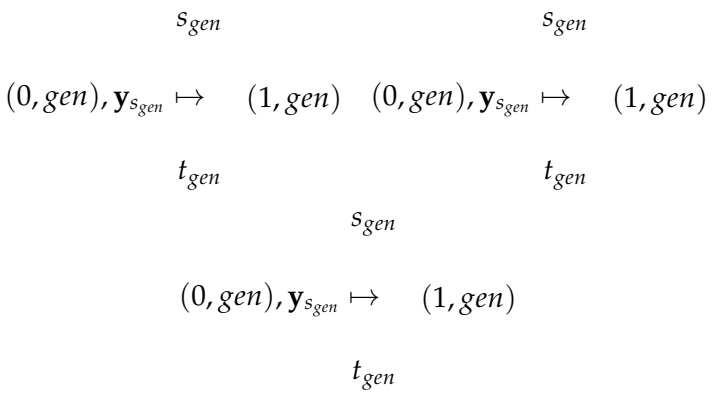


Figure 3.2: Drawing of the graphs $\mathcal{G}$

Note that while this is a DAG combiner implements the same robust $(2;1)$ black-box combiner as in **??**, but not the combiner that would be constructed by the constructive proof below since that is much more tractable. The DAG combiner constructed in the proof is shown in **??**.

Since later on we will focus on showing the impossibility of certain combiner constructions it's essential that the notion of combiner that we use is as strong as possible. By definition a robust black-box DAG combiner is also a robust black-box combiner. We show that the notions of robust black-box DAG combiner and robust black-box combiner are equally strong and capture the same set of constructions. We formulate this result in the following theorem.

Theorem 3.2 (Generality of Robust $(n;k)$ Black-Box DAG Combiners) *For every robust $(n;k)$ black-box combiner there exists a computationally equivalent robust $(n;k)$ black-box DAG combiner.*

Intuition. We need to show that for every robust $(n;k)$ black-box combiner Π we can construct a robuts $(n;k)$ black-box DAG combiner $\widetilde{\Pi}$ that represents the same function if interpreted, given that Π is efficient is also efficient. The proof works by unrolling the computation done by the combiner Π for every oracle call made to a candidate scheme. Since we know that Π is a black-box combiner, it runs in time polynomial in λ, n, m. Let $T \in \mathsf{poly}(\lambda, n, m)$ be an upper bound on the number of steps that the algorithm runs over all inputs $\mathbf{x}$ into the combiner. We start by constructing a graph $\mathcal{G}_{alg}$ that consists of T layers of nm vertices, where n is the number of candidates and m is the number of algorithms implemented by the primitive. On every layer, each vertex corresponds to an oracle call to a specific oracle algorithm of a candidate. We connect two vertices $v, w \in V_{alg}$ if the they are not in the same layer with an edge pointing from the vertex in the layer closer to the source to the other vertex. To define the functions f^v_{alg} for $v \in V_{alg}$, a vertex in layer $k \in [T]$, we construct a Turing machine that simulates the robust black-box combiner Π until the k'th oracle query. Once the $i \in [k-1]$'th oracle query is made we instead use the correct output of the correct vertex in the i'th layer and move on. For the k'th query we simply query the oracle of our vertex if the oracle is correct (and also if its is not correct, this will then simply be ignored later one during the execution).

To prove that this construction results in the same output as the robust black-box combiner does, we proceed by induction on the layer depth of the current query, claiming that the output of the correct vertex at that layer aligns with the query output of the corresponding query in the execution of the robust black-box combiner. Note that the DAG only contains $T \times n \times m$ many vertices and Turing machines f^v_{alg}. Since the computation at every vertex is bounded from above by the time it takes the robust black-box combiner to run and we only make one oracle call at every vertex, the result of every vertex can be computed in time at most T. Overall the time it takes for the robust black-box DAG combiner to run is therefore polynomial in λ, n, m. We provide a formal proof below.

Proof (Theorem 3.2) Let P be a cryptographic primitive Π_{P} with defining algorithms $(p_1, \ldots, p_{|\mathsf{P}|})$, et Π_{P} be a robust $(n;k)$ black-box P-combiner according to Definition 2.12, and let $j \in [|\mathsf{P}|]$ be arbitrary. By Definition 2.12 there exists a polynomial $T \in \mathsf{poly}(n, \lambda, |x|)$ where x is the input to p_j. We construct a directed acyclic graph $\mathcal{G}_j$

$$\mathcal{G}_j = ([n] \times [|\mathsf{P}|] \times [T] \cup \{s, t\}, \{\, ((a_1, b_1, t_1), (a_2, b_2, t_2)) : t_1 < t_2 \,\})$$

We define the coloring by $c(a, b, t) = (a, b)$. We define the Turing machines $\{f_{j,v}\}_{v \in V(\mathcal{G}_j)}$ iteratively starting from the beginning of the computation s, we define $f_{j,v_{j,a,b,1}}$ to be the ppt Turing machine that simulates the $\Pi_{\mathsf{P}}.p_j$ until the first call to its oracle. If that oracle happens to be (a, b) then we end, otherwise write 0 onto the tape. In every subsequent level $f_{j,a,b,t+1}$ simulates the combiner on and answers the queries to the respective oracle with the output of that query ($y_{j,a,b,k}$ for $k \in [t]$) by progressively simulating all previous Turing machines $f_{j,a,b,k}$ for $k \in [t]$. By checking whether after the computation we find a 0 on the tape we can trace the path of queried oracles. We can now continue the simulation of the combiner since we know exactly the state of the tape. The combiner is necessarily efficient since for every the number of vertices in the graph is bounded from above by $nT(n, \lambda, |x|)|\mathsf{P}|$ where $T \in \mathsf{poly}(m, \lambda, |x|)$, we only make exactly one query to any oracle per vertex (which also puts a polynomial bound on the number of oracle calls), the topological ordering can be computed in polynomial time. We are only left to show that the Turing machines $\{f_{j,v}\}_{v \in V(\mathcal{G}_j)}$ are indeed ppt. To this end note that $f_{j,v}$ simulates the black-box combiner Turing machine, which is assumed to be ppt, at most once where it can answer any oracle query by looking up the corresponding output of the layer in time polynomial in n, $|\mathsf{P}|$ and λ. □

3.2 Impossibility of Robust Black-Box VRF-Combiner

We now tackle the main challenge of combining VRFs in a black-box fashion. Specifically we study the possibility of constructing a robust $(n;k)$ black-box combiner Π_{VRF} for the VRF primitive. During this section we will make the additional assumption that the secret key space of all candidate schemes is equivalent (or at least is in bijective correspondence to) $\{0,1\}^{s(\lambda)}$ for s a polynomial in λ. more introduction

Warm Up. As a warm-up we will start by showing the impossibility of a robust $(2;1)$ black-box combiner for VRFs. That is, we show that there exists no efficient (ppt.) Turing machine Π_{VRF} such that for any two oracles $\mathcal{O}_1, \mathcal{O}_2$ implementing a VF, $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$ implements a secure VRF if at least one of the candidates implements a secure VRF. We state the following lemma.

Lemma 3.3 (Robust (2;1) Black-Box VRF Combiner) *There exists no robust $(2;1)$ black-box combiner in the sense of Definition 2.12 for the VRF primitive defined in Definition 2.9.*

Intuition. We must show that for every construction Π_{VRF} implemented by a ppt. Turing machine there exist two oracles $\mathcal{O}_1, \mathcal{O}_2$, of which at least one implements a secure VRF, such that Π_{VRF} is either not pseudorandom in the sense of Definition 2.6, perfectly correct in the sense of Definition 2.4 or not uniquely provable in the sense of Definition 2.5. Applying the result from Theorem 3.2 we can argue about the non-existence of DAG combiners instead, fix an arbitrary DAG combiner Π_{VRF} and consider the DAG corresponding to the evaluation algorithm $\mathcal{G}_{eval}$. We assume that the combiner is uniquely provable and perfectly correct and construct an adversary $\mathcal{A}$ that wins the prf security experiment against the combiner with significant advantage. For that, the adversary will employ the PSPACE oracle PSPACE to compute a set of candidate secret keys by iteratively filtering the secret key space on a polynomial number of evaluation points the adversary obtains in the first stage of the prf experiment. To execute the filtering the adversary constructs a ppt. Turing machine *eval* that takes as input a tuple (sk^*, x), the combiner input, and returns (y, π) such that the output aligns with the combiner output if sk^* is indeed the secret key input and with high probability if the sk^* is a different secret key. Importantly *eval* must not make any queries to the candidate oracles, that way we can use the PSPACE complete oracle and an a tuple (x, y, π) to compute:

$$\mathcal{SK}' = \left\{\mathsf{sk} \in \mathcal{SK} : eval^{\mathsf{PSPACE}}(\mathsf{sk}, x) = (y, \pi)\right\}$$

Assuming that we can efficiently compute *eval* we can refine the set of candidates $2s(\lambda)$ times. Now assume that we eliminate at least half the

candidates in every second iteration, then we end up with a polynomial set of candidates, in which the true secret key is contained. The adversary can now guess the correct secret key with non-negligible probability and win the pseudorandomness experiment with non-negligible advantage. If on the other hand the set of candidates doesnt shrink at half in at least every second step then intuitively on random inputs x a lot of secret keys (whether or not they are the correct secret key) result in the same output of *eval*, which with non-negligible probability agrees with the combiner. We can still simply sample a secret key of this set of candidates to obtain a secret key sk, with possible negligible chance to be the correct secret key, but non-negligible probability to result in the same output y on a random x. This again enables the adversary to beat the prf security experiment with non-negligible advantage.

We now show how to construct the Turing machine *eval*, for that we first fix arbitrary oracles $\mathcal{O}_1, \mathcal{O}_2$ which are uniquely provable and perfectly correct, a secret key sk in the support of the key generation function and any evaluation point x. We now consider the vertices $v \in V_{eval}$ in a fixed topological ordering and classify them for *relevance* in the sense that a vertex is *relevant* if there exist two outputs $\hat{\mathbf{y}}_v \neq \mathbf{y}_v$ in the set of possible outputs of the corresponding oracle query (e.g. $\mathcal{Y}_{\mathcal{O}_1} \times \mathcal{P}_{\mathcal{O}_1}$ if the query is to evaluation of candidate $\mathcal{O}_1$) such that if we simulate the the execution of the combiners evaluation algorithm but replace the reply of the oracle with $\hat{\mathbf{y}}_v$ then we obtain a different output $\hat{\mathbf{y}}_{t_{eval}} \neq \mathbf{y}_{t_{eval}}$. We will for now assume that that every oracle query is only queried once on every distinct input $\mathbf{x}$, we'll show that this is the case later. The same classification of vertices we do for the verification algorithm.

Now initialize the second candidate with $\mathcal{O}_0$ and the first candidate with any secure VRF. To replace the oracle queries in the combiner and arrive at *eval* on a high level we argue that a vertex is relevant in the evaluation algorithm if and only if there exists a relevant vertex in the verification algorithm such that the queries made at that vertex is exactly the same as the one done in eval. Since every relevant query is repeated in the verification algorithm and every not repeated query in the verification is irrelevant we can simply define *eval* to be the Turing machine for (x, y, π, τ) where τ is the verification algorithms transcript on x, y, π that simulates the combiners evaluation algorith, if he encounters an oracle query to $\mathcal{O}_1$ the simulator checks the transcript τ if it contains the answer and answers with the answer if it exists. If it doesnt exist this means that either 1. the secret key doesn't correspond to (x, y, π) or and the query is possibly even relevant or 2. the secret key is correct, or related, and the query is irrelevant. If it encounters a query to $\mathcal{O}_2$ it the answer is known.

If we are in case 2., then we can simply iterate over the vertices in topological ordering and try every value if the query is not in the transcript until

every combination of possibile outputs is tested, obtaining the same output every time. Assume that were not the case then we could again break unique provability by making $\mathcal{O}_2$ secure again (vrf-ification) and break unique provability. If we are in case 1. then and for every guessing through the combiner (except for the recordings in τ) the output is y, then it is also y for the honest evaluation. We conclude that by only allowing those secret keys to survive at every step, we conserve *exactly* all secret keys which evaluate x to y, π.

Now we quickly need to argue that oracle queries are *unique* in the sense that a given algorithm of a given candidate is queried at most once on a specific input $\mathbf{x}$ and that we can construct a secure VRF from a uniquely provable and perfectly correct VRF that agrees on the original VRF on all secret keys in a polynomial set. For uniqueness the case is clear since the oracles need to be deterministic, assume they were not deterministic and that two outputs in the support would result in a different output, than we would arrive at a contradiction since by perfect correctness both must verify since they are honestly generated outputs and by unique provability not both must verify since they are distinct. We can therefore assume that the candidates are deterministic or if they are random we can just replace the random output with an arbitrary but fixed output in the support. The DAG combiner can now be slightly modified to store the all queries made, if a query is made a second time then instead of requerying he could just simply use the result from the first query (since in our DAG construction the Turing machine has access to *all* previous outputs).

For the VRF-ification, we give some intuition why a secure VRF can be remapped on a polynomial set of secret keys to assume the same output as a uniquely provable and perfectly correct VRF. For that remapping we introduce a symmetric and reflexive relation $\sim \subseteq \mathcal{SK} \times \mathcal{SK}$ defined as follows:

$$\mathsf{sk}_1 \sim \mathsf{sk}_2 \iff \exists \mathsf{pk} \in \mathcal{PK}, \Pr\left[\begin{matrix}(\mathsf{sk}_1, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda) \wedge \\ (\mathsf{sk}_2, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\end{matrix}\right] > 0$$

Let $\sim^* := \bigcup_{k \in \mathbb{N}} \sim^k$ be the transitive closure of $\sim$, since $\sim$ is reflexive and symmetric already $\sim^*$ is an equivalence relation. We claim that if two keys $\mathsf{sk}_1, \mathsf{sk}_2$ are in the same equivalence relation, i.e. $\mathsf{sk}_1 \sim^* \mathsf{sk}_2$ then $\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_1, \cdot) = \mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_1, \cdot)$ since otherwise we could break unique provability. If we now want to remap the outputs of a VRF $\mathcal{O}_{\mathbf{VRF}}$ on the set $\widetilde{\mathcal{SK}}$ then we need to remap the outputs of evaluation in $\widehat{\mathcal{SK}} := \bigcup_{\tilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}} [\tilde{\mathsf{sk}}]_{\sim^*}$, the union of all equivalence classes of $\widetilde{\mathcal{SK}}$. Since the newly mapped to points come from a uniquely provable and perfectly correct VF and the secure VRF is also a uniquely provable and perfectly correct VRF the resulting VF is uniquely provable and perfectly correct. To show that its pseudorandom we

show that in the execution of the pseudorandomness game the probability of sampling a secret key $\widehat{\mathsf{sk}} \in \widehat{\mathcal{SK}}$ is negligible if $|\widehat{\mathcal{SK}}| \in \mathsf{poly}(\lambda)$. Assume towards a contradiction that the following holds:

$$\Pr\left[\mathsf{sk} \in \widehat{\mathcal{SK}} : (\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^{\lambda})\right] > \frac{1}{p(\lambda)}$$

For a polynomial $p \in \mathsf{poly}(\lambda)$, then we could simply resample a secret key sk' which would also with probability at least $1/p(\lambda)$ be in $\widehat{\mathcal{SK}}$. Additionally with probability at least $1/|\widehat{\mathcal{SK}}| \in \mathsf{poly}(\lambda)$ we have $\mathsf{sk} \sim^{*} \mathsf{sk}'$ which means that with probability $1/p(\lambda)|\widehat{\mathcal{SK}}| \in \mathsf{poly}(\lambda)$ we obtain a secret key which allows us to recompute all outputs, breaking pseudorandomness.

Proof of Lemma 3.3. Let Π_{VRF} be an arbitrary robust $(2;1)$ black-box DAG combiner for the VRF primitive. We claim that there exists a related robust black-box DAG combiner for the VRF primitive that only queries every implementation function of every candidate scheme on any input $\mathbf{x}$ at most once.

Claim 3.4 *Let Π_{VRF} be a robust $(2;1)$ black-box DAG combiner for the VRF primitive, there exists a robust $(2;1)$ black-box DAG combiner for the VRF primitive $\widetilde{\Pi_{\mathsf{VRF}}}$ such that in its evaluation and verification algorithm DAG $\mathcal{G}_{eval}$ and $\mathcal{G}_{vfy}$ and each of the vertices $v \in V_{eval}$ and $w \in V_{vfy}$ the call to*

The idea is to slightly alter the intermediate Turing machines to check, before they make a query, whether the oracle has already been queried on that specific input. If that is the case then the oracle is queried on a fresh input but the output is ignored. Subsequent queries that would depend on that query re-simulate the execution anyway and therefore know on which value said oracle was queries, they then always refer to the value returned on the topologically first query. Assume that this would change the combiner execution for some query, then this means that the oracle output were not deterministic and based on the internal randomness the combiner output changes. By perfect correctness both resulting distinct outputs must verify which would violate unique provability.

Additionally we show that given two VF oracles $\mathcal{O}_{\mathbf{VRF}}$ and $\mathcal{O}_{up}$ such that $\mathcal{O}_{\mathbf{VRF}}$ implements a secure VRF and $\mathcal{O}_{up}$ implements a perfectly correct and uniquely provable VF we can construct a new oracle $\mathcal{O}_{\mathbf{VRF}}(\mathcal{O}_{up}, \widetilde{\mathcal{SK}})$ such that $\mathcal{O}_{\mathbf{VRF}}(\mathcal{O}_{up}, \widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ implements a secure VRF and its evaluation algorithm agrees on all points (sk, x) with the oracle $\mathcal{O}_{up}$ where $\mathsf{sk} \in \widetilde{\mathcal{SK}}$ and $\mathsf{pk} \in \widetilde{\mathcal{PK}}$ if $|\widetilde{\mathcal{SK}}|, |\widetilde{\mathcal{PK}}| \in \mathsf{poly}(\lambda)$. Formally:

Claim 3.5 *Let $\mathcal{O}_{\mathbf{VRF}}$ be a VF oracle that implements a secure VRF in the sense of Definition 2.9, $\mathcal{O}_{up}$ be a VF oracle that is uniquely provable in the sense of Definition 2.5 and perfectly correct in the sense of Definition 2.4 such that $\mathcal{O}_{\mathbf{VRF}}$ and $\mathcal{O}_{up}$ are on the same spaces and let $\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}$ be polynomially bounded subsets of the secret and public key spaces of both oracles. There exists an oracle $\mathcal{O}_{mix}$ such that the following properties are satisfied:*

1. *The evaluation algorithm of $\mathcal{O}_{mix}$ agrees with the evaluation algorithm of $\mathcal{O}_{up}$ on all secret keys $\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}$, i.e. we have that*

$$\forall \widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}, \mathcal{O}_{mix}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, \cdot) = \mathcal{O}_{up}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, \cdot)$$

2. *The verification algorithm of $\mathcal{O}_{mix}$ agrees with the evaluation algorithm of $\mathcal{O}_{up}$ on all public keys $\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}$, i.e. we have that*

$$\forall \widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}, \mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, \cdot, \cdot, \cdot) = \mathcal{O}_{up}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, \cdot, \cdot, \cdot)$$

3. *The oracle $\mathcal{O}_{mix}$ implements a secure VRF in the sense of Definition 2.9.*

Note first that it is not sufficient to simply change all of the mappings of the evaluation algorithm of $\mathcal{O}_{\mathbf{VRF}}$ on the points in $\widetilde{\mathcal{SK}}$ to the output of $\mathcal{O}_{up}$ and the mappings of the verification algorithm of $\mathcal{O}_{\mathbf{VRF}}$ on the points in $\widetilde{\mathcal{PK}}$ to the output of $\mathcal{O}_{up}$. Such a construction would indeed not necessarily be uniquely provable or perfectly correct since for one we might have not guaranteed that the verification of a newly mapped output succeeds or the verification of a previously mapped output rejects and also we need to consider that if two secret keys have non-zero probability to be sampled with the same public key, then they both need to induce the same mapping $\mathcal{X} \to \mathcal{Y} \times \mathcal{P}$, also this holds for the public key perspective if we change the mappings of the verification algorithm.

Proof of Claim 3.5. Let $\sim_{\mathcal{SK}} \subseteq \mathcal{SK}^2, \sim_{\mathcal{PK}} \subseteq \mathcal{PK}^2$ be relations defined as follows.

$$\mathsf{sk}_1 \sim_{\mathcal{SK}} \mathsf{sk}_2 \iff \exists \mathsf{pk} \in \mathcal{PK}, \Pr\left[\begin{matrix}(\mathsf{sk}_1, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda) \wedge \\ (\mathsf{sk}_2, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\end{matrix}\right] > 0 \tag{3.1}$$

$$\mathsf{pk}_1 \sim_{\mathcal{PK}} \mathsf{pk}_2 \iff \exists \mathsf{sk} \in \mathcal{SK}, \Pr\left[\begin{matrix}(\mathsf{sk}, \mathsf{pk}_1) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda) \wedge \\ (\mathsf{sk}, \mathsf{pk}_2) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\end{matrix}\right] > 0 \tag{3.2}$$

And consider $\sim^* = \bigcup_{k\in\mathbb{N}} \sim^k$, the transitive closure of $\sim$. We claim that $\sim^*$ is an equivalence relation as its the transitive closure of a reflexive and symmetric relation $\sim$. We use $[\mathsf{sk}]_{\sim^*}$ for a secret key $\mathsf{sk} \in \mathcal{SK}$ to denote the equivalence class of sk under $\sim^*$, i.e. $\{\mathsf{sk}' \in \mathcal{SK} : \mathsf{sk} \sim^* \mathsf{sk}'\}$. Additionally for a public key $\mathsf{pk} \in \mathcal{PK}$ define $R_{\mathcal{PK}}(\mathsf{pk})$, the subset of secret keys *related* to pk and symmetrically for a secret key $\mathsf{sk} \in \mathcal{SK}$ define $R_{\mathcal{SK}}(\mathsf{sk})$, the subset of public keys *related* to sk.

$$R_{\mathcal{PK}}(\mathsf{pk}) := \left\{\mathsf{sk} \in \mathcal{SK} : \Pr\left[(\mathsf{sk},\mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\right] > 0\right\} \tag{3.3}$$

$$R_{\mathcal{SK}}(\mathsf{sk}) := \left\{\mathsf{pk} \in \mathcal{PK} : \Pr\left[(\mathsf{sk},\mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\right] > 0\right\} \tag{3.4}$$

We now define the oracle $\mathcal{O}_{mix}$ using the following implementation functions for its corresponding key generation function $\mathcal{O}_{mix}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$, its evaluation function $\mathcal{O}_{mix}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$ and its verification function $\mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi)$ for that we first define the set of secret and public keys $\mathcal{SK}_{map}$ and $\mathcal{PK}_{map}$ respectively on which we need to remap $\mathcal{O}_{\mathbf{VRF}}$.

$$\mathcal{SK}_{map} := \bigcup_{\widetilde{\mathsf{sk}}\in\widetilde{\mathcal{SK}}} [\widetilde{\mathsf{sk}}]_{\sim^*_{\mathcal{SK}}} \cup \bigcup_{\widehat{\mathsf{pk}}\in\bigcup_{\widetilde{\mathsf{pk}}\in\widetilde{\mathcal{PK}}}[\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{PK}}}} R_{\mathcal{PK}}(\widehat{\mathsf{pk}}) \tag{3.5}$$

$$\mathcal{PK}_{map} := \bigcup_{\widetilde{\mathsf{pk}}\in\widetilde{\mathcal{PK}}} [\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{PK}}} \cup \bigcup_{\widehat{\mathsf{sk}}\in\bigcup_{\widetilde{\mathsf{sk}}\in\widetilde{\mathcal{SK}}}[\widetilde{\mathsf{sk}}]_{\sim^*_{\mathcal{SK}}}} R_{\mathcal{SK}}(\widehat{\mathsf{sk}}) \tag{3.6}$$

Now we define $\mathcal{O}_{mix}$ generation to simply be the VRF oracle key generation, we define the evaluation of $\mathcal{O}_{mix}$ to interpolate between the evaluation of $\mathcal{O}_{\mathbf{VRF}}$ and $\mathcal{O}_{up}$ based on whether the secret key is in $\mathcal{SK}_{mix}$ and finally we define the verification of $\mathcal{O}_{mix}$ to interpolate between the verification functions of $\mathcal{O}_{\mathbf{VRF}}$ and $\mathcal{O}_{up}$ based on whether the public key is in $\mathcal{PK}_{mix}$. We show the oracle in **??**.

$$\mathcal{O}_{mix}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) = \begin{cases} \mathcal{O}_{up}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) & \text{if } \mathsf{sk} \in \mathcal{SK}_{map} \\ \mathcal{O}_{\mathbf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) & \text{otherwise} \end{cases} \tag{3.7}$$

$$\mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) = \begin{cases} \mathcal{O}_{up}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) & \text{if } \mathsf{pk} \in \mathcal{PK}_{map} \\ \mathcal{O}_{\mathbf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) & \text{otherwise} \end{cases} \tag{3.8}$$

We first show that $\mathcal{O}_{mix}$ is uniquely provable and perfectly correct. To this end we show that if $(\mathsf{sk},\mathsf{pk}) \leftarrow_{\$} \mathcal{O}_{mix}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$ then we have $\mathsf{sk} \in \mathcal{SK}_{map} \iff \mathsf{pk} \in \mathcal{PK}_{map}$. To show one direction let $\mathsf{sk} \in \mathcal{SK}_{map}$ be arbitrary, we have by definition of we have either of the following two cases:

$$\mathsf{sk} \in \bigcup_{\widetilde{\mathsf{sk}}\in\widetilde{\mathcal{SK}}} [\widetilde{\mathsf{sk}}]_{\sim^*_{\mathcal{SK}}} \vee \mathsf{sk} \in \bigcup_{\widehat{\mathsf{pk}}\in\bigcup_{\widetilde{\mathsf{pk}}\in\widetilde{\mathcal{PK}}}[\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{PK}}}} R_{\mathcal{PK}}(\widehat{\mathsf{pk}})$$

In the first case there exists we have that $\mathsf{pk} \in \bigcup_{\widehat{\mathsf{sk}} \in \bigcup_{\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}} [\widetilde{\mathsf{sk}}]_{\sim^*_{\mathcal{SK}}}} R_{\mathcal{SK}}(\widehat{\mathsf{sk}})$ by definition of $R_{\mathcal{SK}}(\widehat{\mathsf{sk}})$ and therefore also $\mathsf{pk} \in \mathcal{PK}_{map}$. For the other side assume $\mathsf{pk} \in \mathcal{PK}_{map}$ then the symmetric argument holds. In the second case there exists a $\widehat{\mathsf{pk}} \in \bigcup_{\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}} [\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{SK}}}$ such that sk may be sampled with $\widehat{\mathsf{pk}}$, since for this $\widehat{\mathsf{pk}}$ it holds that $\mathsf{pk} \sim^*_{\mathcal{PK}} \widehat{\mathsf{pk}}$ by definition of $\sim^*_{\mathcal{PK}}$ it must holds that also $\mathsf{pk} \in \bigcup_{\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}} [\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{PK}}}$ and therefore also $\mathsf{pk} \in \mathcal{PK}_{map}$.

maybe not smash this

The perfect correcntess follows directly from holds by this property and the fact that both $\mathcal{O}_0, \mathcal{O}_{\mathbf{VRF}}$ are perfectly correct. For unique provability assume that there exist two distinct outputs $\mathbf{y}_0 \neq \mathbf{y}_1$ such that $\mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, \mathbf{y}_0, \pi_0) = \mathcal{O}_{mix}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, \mathbf{y}_1, \pi_1) = 1$, now if $\mathsf{pk} \in \mathcal{PK}_{map}$ we break unique provability of $\mathcal{O}_0$ and if $\mathsf{pk} \notin \mathcal{PK}_{map}$ we break unique provability of $\mathcal{O}_{\mathbf{VRF}}$, therefore the result must be both uniquely provable and perfectly correct.

We show that if $\mathcal{O}_{\mathbf{VRF}}$ is pseudorandom then $\mathcal{O}_{mix}$ is also pseudorandom. Assume, towards a contradiction, that $\mathcal{O}_{mix}$ is not pseudorandom, i.e. that there exists an efficient (ppt.) adversary $\mathcal{A}$ such that $\mathcal{A}$ wins the pseudorandomness experiment against $\mathcal{O}_{mix}$ with non-negligible advantage. We now show that the pseudorandomness game against $\mathcal{O}_{mix}$ and $\mathcal{O}_{\mathbf{VRF}}$ are statistically close, i.e. we bound the probability that the hidden secret key $\mathsf{sk} \in \mathcal{SK}_{map}$ and apply the difference lemma.

define games of the hybrid formally

$$\begin{aligned}
\mathbf{Adv}^{\mathrm{prf}}_{\mathsf{VRF}}(\mathcal{A}, \mathcal{O}_{mix}) &= \left|\Pr\left[H_0^{prf}\right] - \Pr\left[H_1^{prf}\right]\right| && (3.9)\\
&= \left|\Pr\left[H_0^{prf}\right] - \Pr\left[H_2^{prf}\right] + \Pr\left[H_2^{prf}\right] - \Pr\left[H_1^{prf}\right]\right| && (3.10)\\
&\leq \left|\Pr\left[H_0^{prf}\right] - \Pr\left[H_2^{prf}\right]\right| + \left|\Pr\left[H_2^{prf}\right] - \Pr\left[H_1^{prf}\right]\right| && (3.11)\\
&\leq \varepsilon^{prf}_{VRF,mix} + \left|\Pr\left[H_2^{prf}\right] - \Pr\left[H_1^{prf}\right]\right| && (3.12)
\end{aligned}$$

Let $\mathcal{E}$ be the event that $\mathsf{sk} \in \mathcal{SK}_{map}$ for $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathcal{O}_{mix}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$ then:

$$\begin{aligned}
\Pr\left[H_2^{prf} \wedge \neg\mathcal{E}\right] &= \Pr\left[H_2^{prf} \mid \neg\mathcal{E}\right] \Pr[\neg\mathcal{E}] && (3.13)\\
&= \Pr\left[H_1^{prf} \mid \neg\mathcal{E}\right] \Pr[\neg\mathcal{E}] && (3.14)\\
&= \Pr\left[H_1^{prf} \wedge \neg\mathcal{E}\right] && (3.15)
\end{aligned}$$

By the difference lemma $|\Pr[H_1^{prf} \wedge \neg\mathcal{E}] - \Pr[H_2^{prf} \wedge \neg\mathcal{E}]| \leq \Pr[\mathcal{E}]$, since the

difference lemma is folklore, we present the proof in the appendix.

$$\mathbf{Adv}^{\mathrm{prf}}_{\mathsf{VRF}}(\mathcal{A}, \mathcal{O}_{mix}) \leq \varepsilon^{prf}_{VRF,mix} + \Pr\left[\mathsf{sk} \in \mathcal{SK}_{map}\right] \tag{3.16}$$

$$\leq \varepsilon^{prf}_{VRF,mix} + \Pr\left[\mathsf{sk} \in \bigcup_{\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}} [\widetilde{\mathsf{sk}}]_{\sim^*_{\mathcal{SK}}}\right] \tag{3.17}$$

$$+ \Pr\left[\mathsf{sk} \in \bigcup_{\widehat{\mathsf{pk}} \in \bigcup_{\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}} [\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{PK}}}} R_{\mathcal{PK}}(\widehat{\mathsf{pk}})\right] \tag{3.18}$$

$$\leq \varepsilon^{prf}_{VRF,mix} + \mu_1(\lambda) + \mu_2(\lambda) \tag{3.19}$$

Where $\mu_1, \mu_2 \in \mathsf{negl}(\lambda)$. To show that this use that $|\widetilde{\mathcal{SK}}|, |\widetilde{\mathcal{PK}}| \in \mathsf{poly}(\lambda)$, first for μ_1 assume that $\mu_1 \in \mathsf{poly}(\lambda)$, then an adversary could simply sample a secret key $\mathsf{sk}' \in \widetilde{\mathcal{SK}}$, since sk is in an equivalence class of an element of $\widetilde{\mathcal{SK}}$ we hit that with proabbility at least $1/|\widetilde{\mathcal{SK}}|$ and then we win the game with probability at least $1/p(\lambda)|\widetilde{\mathcal{SK}}|$, hence $\mathcal{O}_{\mathbf{VRF}}$ would not be prf secure, a contradiction. For μ_2 if the secret key being sampled according to $\mathcal{O}_{\mathbf{VRF}}.\mathsf{Gen}_{\mathsf{VRF}}$ is in this set with significant probability, then we can just simply resample to obtain another key with significant probability. Note that due to the polynomial bound on public key-equivalence classes we only have a polynomial number of different evaluations induced by all secret keys in here.

formulate out

Using this result we claim that for a fixed oracle $\mathcal{O}_i$ and any subset of queries made during the evaluation algorithm which are not reflected in the trace of the verification execution, we call such queries *hidden*, the output of the evaluation algorithm doesn't change for all possible outputs of the oracles. That is, if we execute the evaluation algorithm and replace every oracle query to an algorithm of $\mathcal{O}_i$ with the output in the transcript or any other random output in the support if it doesnt occur in the transcript we arrive at the same value for the right secret key sk.

Claim 3.6 *Let $\mathcal{O}_1, \mathcal{O}_2$ be uniquely provable and perfectly correct VF oracles and let S be a set of hidden vertices in G_{eval} with queries made to only one oracle. Then they are jointly irrelevant to the output of the combiners evaluation algorithm. Specifically we can simulate the output of the individual oracle queries by choosing arbitrary elements in the support of the oracles algorithm.*

Proof of Claim 3.6 Assume this were not the case, we follow the topological ordering and initialize the oracle responses to those queries with arbitrary outputs in the support. If at one point the output of the combiner changes then simply remap the corresponding oracle and make the other oracle a

VRF. Subsequently the combiner needs to be uniquely provable, perfectly correct and pseudorandom. By perfect correctness we have

$$\Pi_{\mathsf{VRF}}^{\hat{\mathcal{O}}_1,\mathcal{O}_2^{\mathbf{VRF}}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk},x,\hat{y},\hat{\pi}) = 1 \tag{3.20}$$

$$\Pi_{\mathsf{VRF}}^{\tilde{\mathcal{O}}_1,\mathcal{O}_2^{\mathbf{VRF}}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk},x,\tilde{y},\tilde{\pi}) = 1 \tag{3.21}$$

And by unique provability and $\hat{y} \neq \tilde{y}$ we have that

$$\Pi_{\mathsf{VRF}}^{\tilde{\mathcal{O}}_1,\mathcal{O}_2^{\mathbf{VRF}}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk},x,\tilde{y},\tilde{\pi}) = 0 \tag{3.22}$$

$$\Pi_{\mathsf{VRF}}^{\hat{\mathcal{O}}_1,\mathcal{O}_2^{\mathbf{VRF}}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk},x,\hat{y},\hat{\pi}) = 0 \tag{3.23}$$

Now since the oracle $\tilde{\mathcal{O}}_1$ and $\hat{\mathcal{O}}_1$ only differ in the evaluations not covered by the verification algorithm the execution of the algorithm will be identical, given the input is the same. This fundamentally contradicts (3.21) and (**??**), we conclude that the resulting combiner is either not perfectly correct or uniquely provable.

We can now formulate the adversary $\mathcal{D}$ that distinguishes the random world from the real world in the prf security experiment. First the adversary computes a tuple (x,y,π,τ) computed as follows

$$x \leftarrow_{\$} \{0,1\}^{\lambda} \tag{3.24}$$

$$(y,\pi) \leftarrow \mathcal{O}_{eval}^{prf}(x) \tag{3.25}$$

$$\tau \leftarrow \mathsf{trace}_{\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk},x,y,\pi)} \tag{3.26}$$

following it uses the PSPACE oracle to preform a round of secret key filtering, iterating over all possible secret keys and simulating evaluation for every secret key (without querying any candidate oracles of course). In the simulation it simulates the Turing machine implementing the combiners evaluation algorithm, once an oracle query to a candidate would be made we can perfectly simulate it if its a query to $\mathcal{O}_{\perp}$ or not hidden, i.e. found in τ. If its not found in τ and to oracle $\mathcal{O}_1$ then we iterate over all possible values (either in $\mathcal{SK}\times\mathcal{PK},\mathcal{Y}\times\mathcal{P}$ or $\{0,1\}$) and recurse on every single output. If in the end all evaluations evaluate to y then keep that secret key. Otherwise there exists an execution that results in a different output $y' \neq y$, those candidate secret keys are dropped. Formally we define the following language $\mathcal{L}$

$$\mathcal{L}_{filter} := \left\{ (\alpha,(\mathbf{y}^*,\mathbf{x}^*,\{(\mathbf{x}_i,\mathbf{y}_i,\pi_i,\tau_i)\}_{i\in 2s(\lambda)},\rho)) : \begin{array}{r} \mathsf{sk}^* = \mathcal{S}(\mathcal{SK},\{(\mathbf{x}_i,\mathbf{y}_i,\pi_i,\tau_i)\}_{i\in 2s(\lambda)},\rho) \\ \alpha \text{ is prefix of } \mathsf{sk}^* \end{array} \right\} \tag{3.27}$$

unclear how to formulate this

Formally we define the simulator oracle Turing machine $\mathcal{S}^{\mathsf{PSPACE}}(\mathcal{SK}_i,x,y,\pi,\tau)$ $\mathcal{SK}_{i+1}$ with access to a PSPACE oracle.

I assume to be able to VRFify the uniquely provable and perfectly correct VF. However if I assume to have $\mathcal{O}_{\mathbf{VRF}}$ then why cant the combiner simply initialize itself with $\mathcal{O}_{\mathbf{VRF}}$.

Now we initialize the combiner with the oracles $\mathcal{O}_0, \mathcal{O}_{\mathbf{VRF}}$. In a single filtering process the adversary will iterate over all candidate secret keys, and simulate the execution of the combiner by answering any query to $\mathcal{O}_0$ with 0, and for every query in $\mathcal{O}_{\mathbf{VRF}}$ he checks the transcript and answers with an existing query in there if it exists, and otherwise iterates over all possible values (e.g. $\mathcal{Y} \times \mathcal{P}$ if the query is an evaluation) to continue until the output is computed. If for all evaluations the simulated combiner evaluates to $\mathbf{y}$ then we retain the secret key candidate, if for some evaluation the output evaluates to some other value $\mathbf{y}'$ then we discard it.

We argue that a secret key sk is accepted only if the evaluation of the real combiner (with oracle queries) results in the output $\mathbf{y}$. Assume an sk would not evaluate to $\mathbf{y}$ under a candidate sk, then during all evaluations there exists at least one evaluation (the one corresponding to the real secret key) which doesnt evaluate to $\mathbf{y}$. On the same hand for the correct secret key sk the output will always be $\mathbf{y}$, assume that were not the case, that would mean that for some execution we would have needed to guess a subset of oracle queries which did change the output (the 0 guesses to oracle 1 area always right and the lookups in the transcript are also always right). Then however we could simply modify all those outputs under this secret key in the ignored queries to fit that configuration, breaking unique provability. soundness check

Finally since we have shown that the correct secret key will not be thrown out and only secret keys which evaluate to the correct $\mathbf{y}$ are retained we argue that after $2s(\lambda)$ for any uniformly random sample of secret key of the remaining candidates, the probability of sampling a secret key sk' such that on a uniformly random value x' the output diverges from the true output is negligible. The adversary $\mathcal{D}$ can therefore simply do that to break pseudorandomness with high probability. We show pseudocode for $\mathcal{D}$ in Figure 3.3.

Distinguisher $\mathcal{D}_1^{\mathsf{PSPACE},\mathcal{O}_{eval}^{prf}}(\mathsf{pk})$

$\mathcal{SK}_0 \leftarrow \mathcal{SK}$
for $i \in [2s(\lambda)]$ **do**
  $\mathbf{x}_i \leftarrow_\$ \{0,1\}^\lambda$
  $(\mathbf{y}_i, \pi_i) \leftarrow \mathcal{O}_{eval}^{prf}(\mathbf{x}_i)$
  $\tau_i \leftarrow \mathsf{trace}_{\Pi_{\mathsf{VRF}}^{\mathcal{O}_0,\mathcal{O}_{\mathbf{VRF}}}.\mathsf{Vfy}_{\mathsf{VRF}}}(\mathbf{x}_i, \mathbf{y}_i, \pi_i)$
  // Use PSPACE oracle for the following
  $v_{eval}^1, \ldots, v_{eval}^{|\mathcal{V}_{eval}|} \leftarrow \mathsf{topo}(\mathcal{V}_{eval})$
$\sigma \leftarrow_\$ \mathcal{SK}_{2s(\lambda)}$
$\mathbf{x} \leftarrow_\$ \{0,1\}^\lambda$
return $(\sigma, \mathbf{x})$

Distinguisher $\mathcal{D}_2^{\mathsf{PSPACE},\mathcal{O}_{eval}^{prf}}(\mathsf{pk}, \sigma, \mathbf{x}, \mathbf{y})$

$\mathbf{y}^* \leftarrow \Pi_{\mathsf{VRF}}^{\mathcal{O}_0,\mathcal{O}_{\mathbf{VRF}}}.\mathsf{Eval}_{\mathsf{VRF}}(\sigma, \mathbf{x})$
return $\mathbf{y}^* = \mathbf{y}$

Figure 3.3: Distinguisher.

While robust $(2;1)$ black-box combiners are generally sufficient to construct robust $(n;1)$ black-box combiners, non-existence of a robust $(2;1)$ black-box combiner only allows one to deduce the impossibility of a robust $(n;\lfloor n/2 \rfloor)$ black-box combiner. This holds since we can easily construct a robust $(2;1)$ black-box combiner from a robust $(n;\lfloor n/2 \rfloor)$ black-box combiner by running the robust $(n;\lfloor n/2 \rfloor)$ black-box combiner and for every query made to the first $\lfloor n/2 \rfloor$ oracle candidates we answer with the first oracle candidate given to $\Pi_{\mathsf{VRF}}{}^{(2;1)}$ and for the upper $n - \lfloor n/2 \rfloor$ candidate we simply query the other oracle. If at least one oracle given to $\Pi_{\mathsf{VRF}}{}^{(2;1)}$ implements a secure VRF, then at least $\lfloor n/2 \rfloor$ candidates correspond to a VRF oracle. We can derive the following corollary.

change title to lemma maybe

Lemma 3.7 (Robust $(n;\lfloor n/2 \rfloor)$ Black-Box VRF Combiner) *There exists no robust $(n;\lfloor n/2 \rfloor)$ black-box combiner in the sense of Definition 2.12 for the VRF primitive defined in Definition 2.9 for all $n \in \mathbb{N}^+$.*

Extending this result to an impossibility of robust $(n;n-1)$ seems difficult. We elaborate on the reason why the proof for Lemma 3.3 cannot easily be extended to general robust $(n;n-1)$ black-box VRF-combiners. We still believe that robust $(n;n-1)$ black-box VRF-combiners are not possible and therefore pose the following conjecture.

Conjecture 3.8 *There exists no robust $(n;n-1)$ black-box combiner in the sense of Definition 2.12 for the VRF primitive as defined in Definition 2.9 for all natural numbers $n > 1$.*

instead we will show that even if we relax the requirements of the combiner to implement only weak unpredictability, statistical correctness and statistical

unique provability, there still doesnt exist a combiner, even if we assume that all candidates to be pseudorandom and at least one candidate is perfectly correct and perfectly uniquely provable.

Theorem 3.9 (Weak Robust $(n; \lfloor n/2 \rfloor)$ Black-Box VRF Combiners) *There exists no robust $(n; \lfloor n/2 \rfloor)$ black-box combiner in the sense of Definition 2.12 that implements a VF that is statistically correct in the sense of Definition 2.4, statistically uniquely provable in the sense of Definition 2.5 and weakly unpredictable in the sense of Definition 2.8, even if we assume all candidate schemes to be pseudorandom in the sense of Definition 2.6 and at least one candidate to be perfectly uniquely provable in the sense of Definition 2.5 and perfectly correct in the sense of Definition 2.4.*

Intuition. The proof works similar to the proof in the warm-up, note that the adversary there is already capable to break wVRF security. To preserve the argument even if all candidates are guaranteed to be pseudorandom we can apply a similar approach to VRF-ifiifcation, PRF-ification, that simply remaps a VF on all but a polynomially sized set $\widetilde{\mathcal{SK}}$ and $\widetilde{\mathcal{PK}}$. To even exclude statistically correct and statistically uniquely provable VRFs we only need to weaken the claim that binds hidden queries to verification-irrelevant queries. We use that still, with overwhelming probability, the set of hidden and irrelevant queries align, if that were not the case then with significant probability over (sk, x) we could break unique provability and correctness. This is overwhelming for the attack since the secret key sk is sampled once, and that favourably for the adversary with overwhelming chance, and for the values of x we can bound the probability that we see (sk, x) where this alignment doesnt happen from above by a negligible function.

Proof of Theorem 3.9 TODO

Since the typical notions of VUF and wVRF are all weaker than wVUFs as a corollary we can exclude robust $(n; \lfloor n/2 \rfloor)$ black-box combiners for those respective primitives. We formulate those results as corollaries below:

Corollary 3.10 (Robust $(n; \lfloor n/2 \rfloor)$ Black-Box VUF-combiner) *Given n secure VRF candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$ there exists no robust black-box combiner that implements a secure VRF scheme if even one candidate scheme is not uniquely provable.*

Corollary 3.11 (Robust $(n; \lfloor n/2 \rfloor)$ Black-Box VUF-combiner) *Given n secure VUF candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$ there exists no robust black-box combiner that implements a secure VUF scheme if even one candidate scheme is not uniquely provable.*

3.3 Strong Robust Black-Box Combiners

Having ruled out the possibility of combining VRFs, VUFs, wVRFs even for statistical notions of correctness and unique provability and all oracles pseudorandomness we need to make an even stronger assumption to allow for combiners to exist. We will assume that all candidates satisfy unique provability. This is a strong assumption since unique provability is one of the main properties of VRFs which distinguishes VRFs from PRFs which admit simple robust $(n;1)$ black-box combiner constructions. We note that unique provability is generally required to be perfect for good constructions, meaning that most existing constructions come with rigorous mathematical proofs that assert their unique provability. Examples of this are ECVRF and Lattice VRFs, which are the most famous constructions of VRFs. change

As previously mentioned, PRFs admit a simple robust $(n;1)$ black-box combiner which simply xors the outputs of all PRF candidates. We show that this construction can be extended to VRFs by simply computing the trace of the evaluation algorithms trace and providing this as a proof. The existence of such combiners is formalized in the following lemma.

Lemma 3.12 (Strong Robust $(n;1)$ Black-Box VRF Combiners) *There exist robust $(n;1)$ black-box combiners in the sense of Definition 2.12 for the VRF primitive Definition 2.9 if we assume that all candidates are uniquely provable and perfectly correct.*

Proof of Lemma 3.12.

Claim 3.13 *The combiner in Figure 3.4 is perfectly correct in the sense of Definition 2.4 and uniquely provable in the sense of Definition 2.5 if all candidates are perfectly correct Definition 2.4 and uniquely provable Definition 2.5.*

The proof of this claim follows essentially by definition of the combiner construction and the assumed unique provability and pseudorandomness of all candidates. We lead the detailed argument in the appendix but believe that its easy to see why the claim holds. We only need to show that the combiner is pseudorandom, we employ a reductionist approach to show the following claim.

Claim 3.14 *The combiner in Figure 3.4 is (t,ε)-pseudorandom in the sense of Definition 2.6 if at least one candidates are (t,ε)-pseudorandom. In particular if at least one candidate is pseudorandom then the combiner is pseudorandom.*

Assume, towards a contradiction, that there exists an adversary $\mathcal{A}$ running in time t that breaks pseudorandomness of the combiner $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$ with advantage ε, we show that we can construct an adversary $\mathcal{R}_i$ running in time t_i that wins the prf security experiment of every candidate with advantage

Algorithm $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$

1 : $(\mathsf{sk}_1, \mathsf{pk}_1) \leftarrow_\$ \mathcal{O}_1.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
2 : $(\mathsf{sk}_2, \mathsf{pk}_2) \leftarrow_\$ \mathcal{O}_2.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
3 : **return** $((\mathsf{sk}_1, \mathsf{sk}_2), (\mathsf{pk}_1, \mathsf{pk}_2))$

Algorithm $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$

1 : $(\mathsf{sk}_1, \mathsf{sk}_2) \leftarrow \mathsf{sk}$
2 : $(y_1, \pi_1) \leftarrow \mathcal{O}_1.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_1, x)$
3 : $(y_2, \pi_2) \leftarrow \mathcal{O}_2.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_2, x)$
4 : $y \leftarrow y_1 \oplus y_2$
5 : $\pi \leftarrow (y_1, \pi_1, y_2, \pi_2)$
6 : **return** (y, π)

Algorithm $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi)$

1 : $(\mathsf{pk}_1, \mathsf{pk}_2) \leftarrow \mathsf{pk}$
2 : $(y_1, \pi_1, y_2, \pi_2) \leftarrow \pi$
3 : **req** $y = y_1 \oplus y_2$
4 : **req** $\mathcal{O}_1.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_1, x, y_1, \pi_1) = 1$
5 : **req** $\mathcal{O}_2.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_2, x, y_2, \pi_2) = 1$

Figure 3.4: Combiner for VRFs if all candidate schemes are uniquely provable.

ε_i. The reduction $\mathcal{R}_i$ simulates all the remaining candidates queries and uses adversary $\mathcal{A}$ to break that one candidate. We show the reduction $\mathcal{R}_i$ in Figure 3.5.

Reduction $\mathcal{R}_i^{\mathcal{O}_{eval,i}^{prf}}(\mathsf{pk}_i)$

1 : $b \leftarrow_\$ \{0,1\}$
2 : $(\mathsf{sk}_{1-i}, \mathsf{pk}_{1-i}) \leftarrow_\$ \mathcal{O}_{eval,1-i}^{prf}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
3 : $\mathsf{sk} \leftarrow (\mathsf{sk}_0, \mathsf{sk}_1)$
4 : $\mathsf{pk} \leftarrow (\mathsf{pk}_0, \mathsf{pk}_1)$
5 :

Figure 3.5: Reduction.

finish the reduction

Having shown that general notion of combiners on VRFs even with relaxed requirements on the security properties of the combiner are impossibile and the combiners on VRFs with guaranteed unique provability are easy we want to strengthen the result by considering combiners for VRFs (assuming all candidates are uniquely provable) with advanced additional properties, namely constrainability as in **??**, key-homomorphism as in **??**, aggregability as in **??** and finally unbiasability as in **??**.

For the notions of constrainability, key-homomorphism and aggregability this question is quckly answered as we we show how the above construction of extended PRF combiner can be minimally adapted to combine the notions. We will quickly explain how to extend the construction of the extended PRF combiner to capture those notions and move the rigorous proofs in the appendix.

Constrainable VRF. A constrainable VRF as introduced by [**?**] is a VRF with an additional algorithm $\mathsf{Constrain}_{\mathsf{VRF}}$ that allows the constraining of specific secret keys sk to a subset of evaluation points $S \subseteq \mathcal{X}$, sk_S. We require the additional security property that constrainable keys sk_S cannot be used to evaluate points outside of S and the notion of constraint hiding. Constraint hiding requires that the proof of an evaluation doesn't give away the key used. The robust $(n;1)$ black-box combiner implements the algorithms $\mathsf{Gen}_{\mathsf{VRF}}, \mathsf{Eval}_{\mathsf{VRF}}$ and $\mathsf{Vfy}_{\mathsf{VRF}}$ the same as in Figure 3.4 and implements the constraining algorithm by constraining every candidate secret key in the tuples seperately. The algorithm is shown in **??**. The existence is formally stated in the following lemma with the proof, as mentioned above, deferred to the appendix.

Lemma 3.15 (Strong Robust $(n;n-1)$ Black-Box cVRF-Combiner)

Key-homomorphic VRF. A key-homomorphic VRF as introduced in [**?**] is a VRF for which the secret key space $\mathcal{SK}$ and the output space $\mathcal{Y} \times \mathcal{P}$ admit group structures $(\mathcal{SK}, \otimes_{\mathcal{SK}})$ and $(\mathcal{Y} \times \mathcal{P}, \otimes_{\mathcal{Y}\times\mathcal{P}})$ and satisfies that the evaluation function for a fixed $x \in \{0,1\}^\lambda$ is a homomorphism $\mathsf{Eval}_{\mathsf{VRF}}(x,\cdot) : \mathcal{SK} \to \mathcal{Y} \times \mathcal{P}$. The homomorphism property can be checked by using efficient homomorphism test on groups by the BLR test, which states that only with a constrant number of queries we can show that a function is a homomorphism except with negligible probability. Since this test is global it is executed by the verifier and the evaluator, non-homoomrphic candidates are excluded. The result is stated in the lemma below.

Lemma 3.16 (Strong Robust $(n;n-1)$ Black-Box khVRF-Combiner)

Aggregable VRF. An aggregable VRF is similar to a key-homomorphic VRF but enabling the aggregation of image-proof pairs through in a more general notion.

Unbiasable VRF. An unbiasable VRF requires security against biasing the output of the VRF for maliciously crafted secret keys.

Only touch if sufficient time

3.4 Discussion and Extensions

There are different ways to combine VRFs, for both unique provability and key-homomorphism they cannot be combined in a black-box fashion if we want to preserve properties such as unpredictability and therefore also randomness. On the other hand we manage to construct a robust black-box combiner that combines unbiasabile VRFs under guaranteed unique provability for all candidates. We find that the black-box combination of information theoretic properties generally seems hard if other properties such as unpredictability need to be preserved. While the class of all robust black-box combiners initially seems to be very broad we show that we can capture all polynomially bounded combiners using the notion of a robust black-box DAG combiner, which allows a more systematic analysis of possible constructions by abstracting a graph structure in the computation executed by the implemented algorithms. We have not considered possible combiner constructions for stronger relaxations, such as computational unique provability or correctness with error, this is mainly because the resulting construction only offer weak guarantees that are insufficient for practical applications.

We show that there exists no robust $(n; \lfloor n/2 \rfloor)$ black-box combiner for the VRF primitive, naturally we wonder whether an impossibility can be shown for robust $(n; n-1)$ black-box VRF-combiners which eliminates the possibility of combining VRFs if even one candidate is not guaranteed to be secure. We believe this statement to be true, at least under standard assumptions, but proving this seems to be non-trivial.

Chapter 4

Conclusion

Bibliography

[GGM86] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, October 1986.

[MRV99] Silvio Micali, Michael O. Rabin, and Salil P. Vadhan. Verifiable random functions. pages 120–130, 1999.

Appendix

The Difference Lemma

We present the proof of the difference lemma, formally:

Lemma .1 (Difference Lemma) *Let $\mathcal{W}_0, \mathcal{W}_1, \mathcal{E}$ be events with*

$$\Pr[\mathcal{W}_0 \wedge \neg\mathcal{E}] = \Pr[\mathcal{W}_1 \wedge \neg\mathcal{E}]$$

Then we can bound the difference in probability of $\mathcal{W}_0$ and $\mathcal{W}_1$ with

$$|\Pr[\mathcal{W}_0] - \Pr[\mathcal{W}_1]| \leq \Pr[\mathcal{E}]$$

Proof of Lemma .1. Let $\mathcal{W}_0, \mathcal{W}_1, \mathcal{E}$ be events and assume $\Pr[\mathcal{W}_0 \wedge \neg\mathcal{E}] = \Pr[\mathcal{W}_1 \wedge \neg\mathcal{E}]$. Then we can deduce the following:

$$\begin{aligned}
&|\Pr[\mathcal{W}_0] - \Pr[\mathcal{W}_1]| && (.1)\\
=&|\Pr[\mathcal{W}_0 \wedge \mathcal{E}] + \Pr[\mathcal{W}_0 \wedge \neg\mathcal{E}] - \Pr[\mathcal{W}_1 \wedge \mathcal{E}] - \Pr[\mathcal{W}_1 \wedge \neg\mathcal{E}]| && (.2)\\
=&|\Pr[\mathcal{W}_0 \wedge \mathcal{E}] - \Pr[\mathcal{W}_1 \wedge \mathcal{E}]| && (.3)\\
=&|\Pr[\mathcal{W}_0 \mid \mathcal{E}]\Pr[\mathcal{E}] - \Pr[\mathcal{W}_1 \mid \mathcal{E}]\Pr[\mathcal{E}]| && (.4)\\
=&|\Pr[\mathcal{W}_0 \mid \mathcal{E}] - \Pr[\mathcal{W}_1 \mid \mathcal{E}]|\Pr[\mathcal{E}] && (.5)\\
\leq&\Pr[\mathcal{E}] && (.6)
\end{aligned}$$

It is sufficient to find a $(1;2)$-robust black-box combiner for VRF where all candidates are uniquely provable. Given a combiner we can then construct a $(1;n)$-robust black-box combiner for VRF by nesting the combiners iteratively. A construction for the combiner is given in **??**. The key generation works by querying both candidates on the given security parameter and grouping the secret and public keys respectively. Evaluation again queries both candidate schemes on the input with their respective secret key and returns the xor of the outputs while the the individual outputs and proofs consistute the new proof. The verification algorithm checks whether the image y is the result of

xoring the individual outputs of the candidates and whether the individual images y_1, y_2 verify with respect to the verification key the proof and the pre-image. To ensure that the combiner is still efficient we sim

The combiner is clearly efficient since every algorithm consists of one call to the respective algorithm of each candidate scheme (whi)

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	**First name(s):**

With my signature I confirm the following:
- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	**Signature(s)**

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).